

Phase 1)

```
abheetha@annanya:~/PES1UG24AM045-pes-vcs1$ make test_objects
make: 'test_objects' is up to date.
abheetha@annanya:~/PES1UG24AM045-pes-vcs1$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

```
abheetha@annanya:~/PES1UG24AM045-pes-vcs1$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

Phase 2)

```
abheetha@annanya:~/PES1UG24AM045-pes-vcs1$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

```
abheetha@annanya:~/PES1UG24AM045-pes-vcs1$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9a
afc12 | head -20
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050   blob 16.Hello, P
00000010: 4553 2d56 4353 210a                   ES-VCS!.
```

Phase 3)

```
Initialized empty PES repository in .pes/
abheetha@annanya:~/PES1UG24AM045-pes-vcs1$ echo "hello" > file1.txt
abheetha@annanya:~/PES1UG24AM045-pes-vcs1$ echo "world" > file2.txt
abheetha@annanya:~/PES1UG24AM045-pes-vcs1$ ./pes add file1.txt
abheetha@annanya:~/PES1UG24AM045-pes-vcs1$ ./pes add file2.txt
abheetha@annanya:~/PES1UG24AM045-pes-vcs1$ ./pes status
Staged changes:
  staged:   file1.txt
  staged:   file2.txt

Unstaged changes:
(nothing to show)

Untracked files:
  untracked: test_tree.c
  untracked: commit.c
  untracked: index.h
  untracked: tree.c
  untracked: tree.h
  untracked: pes.h
  untracked: test_objects.c
  untracked: pes.c
  untracked: test_sequence.sh
  untracked: .gitignore
  untracked: commit.h
  untracked: README_ProofOfExecution.md
  untracked: object.c
  untracked: Makefile
  untracked: index.c
  untracked: README.md
```

```
abheetha@annanya:~/PES1UG24AM045-pes-vcs1$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776765586 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776765597 6 file2.txt
```

Phase 4)

```
abheetha@annanya:~/PES1UG24AM045-pes-vcs1$ echo "Hello" > hello.txt
abheetha@annanya:~/PES1UG24AM045-pes-vcs1$ ./pes add hello.txt
./pes commit -m "Initial commit"
Committed: 9631bfd2025f... Initial commit
abheetha@annanya:~/PES1UG24AM045-pes-vcs1$ echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"
Committed: e246e35bf776... Add world
abheetha@annanya:~/PES1UG24AM045-pes-vcs1$ echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"
Committed: ab92c228589c... Add farewell
abheetha@annanya:~/PES1UG24AM045-pes-vcs1$ ./pes log
commit ab92c228589c11102bd2b4c47f866ab323f91bac59bd992355197e938e267ab7
Author: PES User
Date: 0

commit e246e35bf77689ddbdae1fc882a3a87cdf6179a0c3e82eb358d73e204d1947d4
Author: PES User
Date: 0

Add farewell

commit 9631bfd2025f574575012f7cf7dffff20b94acd450bdc1dddf83a9e7e276430223
Author: PES User
Date: 0
```

```
.pes/HEAD
.pes/index
.pes/objects/02/d663c03330e7ec172a16b38b8bb05de3ac3553489f2c7d0065a69413cea149
.pes/objects/05/93741b9919d9928d2b3ba750017932a42a8191cc5966ceb4a75f20c171784d
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/06/604861a834e4ea1fcdcf8a8637979f22d3304b857d7b2887fdcfc877596448
.pes/objects/0f/d461a199aa180259f80ed6903c642e92e7b24e9994c757d3ad8ec86754781d
.pes/objects/10/728aa21f3339d1acad60aff88ea409783e28aab93fcb223c89dade5365889a
.pes/objects/10/d488e897a9291aca9f7f55db526cf1c0a4d2a6274de8d6729d8da036cce9fd
.pes/objects/1e/16f4eff3ece225f2698d0cd5aeb58676816db9037014a55574892f11bc028
.pes/objects/1f/f2273de695f67d7f76daf405ef20d448c5d806eff0d139829795d5a94dd56f
.pes/objects/23/93dd1b5b22510ca4aac4b6fffc654a598fe27475228c9a19cf5cd85e5cccd5e
.pes/objects/2c/9f125b9c0764887ba7dbef2c5901a2a26aff1a83bfad89555888d82f84f366
.pes/objects/2c/f8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4
.pes/objects/34/3d34bf690fefa64dc0d95a01280df0e5d489c8eced213340be03ea5ffacc2b
.pes/objects/3d/d9f832ab4ce50c29641e8fdb7bc1c4f5a4710008008ce332e25857671d85ac
.pes/objects/42/359f28504398314d1dfe01f0ef7e0c9ccdfad2db61c8e6232ed7fd3ff0bfc
.pes/objects/43/fa80f75dc47636262c85b7204e521fb8120afff22b104656e17b3eb02d9b12
.pes/objects/47/3a0f4c3be8a93681a267e3b1e9a7dcda1185436fe141f7749120a303721813
.pes/objects/4e/ee9bf43bce4a2510b720c3672a0505298de4fb3606c1d7be42b3b5d88c7442
.pes/objects/59/d5de5d3efcdc85318705e6d6a6d6007dda02a97cf7936fec012ee0638cd567
.pes/objects/5c/7cb7ef91f7a46b8f1014960845f5d6ec6c696bda1aa9378e92711d86e5a23c
.pes/objects/5c/8a64898b18404d965d0ce601571cb23b1bfa39d487f5f79d22192e5d0dd39
.pes/objects/5c/b4602d894715af84f4b654b8cb28db957bcdabab5b72a4cd52234521bf3bc7f
.pes/objects/5f/3340bd3e869ab61fb495f58eb61d87d4c886431c5bb9317d582b01b27e8673
.pes/objects/60/459622a6a99c40ddfc668e7cbd0d70a4d53aeb448a251aa982ea2043e3838
.pes/objects/62/ed9a292d72f52ac600ca97bbaa5abad223349e5d69f237e385b0ed3a49da0f
```

```
.pes/objects/96/31bfd2025f574575012f7cf7dfff20b94acd450bdc1ddf83a9e7e276430223
.pes/objects/97/9f31f30023b39f2ee5b19584079c71d3c5aa9c16a7fea623dc2fb9a16b3d0e
.pes/objects/9c/983f68c966f325edacfc60231399ec84a9a9f2522c8b86ff84ac04dceb4209
.pes/objects/9e/91a910315ee6c14d78424a59599647c349d3de08186f691642258f5f1afedd
.pes/objects/a7/4c8d33a8cdae4949c89c9996cf1d246bbd04c40826d9dc7d25fbc611236b73
.pes/objects/a7/573b95a28adc55e8b33d49407202178b0008faf4ecc39495e2e8737883a4dd
.pes/objects/ab/92c228589c11102bd2b4c47f866ab323f91bac59bd992355197e938e267ab7
.pes/objects/b5/6bb43becbe211f8f0ce7ebb7453cda1cb47986ae37f48aba463cae45f41241
.pes/objects/b9/8a60fc3658cf18c8264d999b2893ea1231b5047406b717068bb59478a82404
.pes/objects/b9/9f25b5b7ce8abc32c8255ce7657c1a9f0f68e48f24f2ef9bcb9cda94bac19f
.pes/objects/c2/80a4927153bb808eb93c61b64948cc2763d55f07a1c0274b6799130b106ab0
.pes/objects/c6/1c81f05a73d23b655a66a0353ff9058d0fcc64bc79fad3f2f5b86999329d41
.pes/objects/c9/ed899783ff70cac61dbbc4e205d04714402c853c4e5f6107d2c8b21cd3a8f4
.pes/objects/ca/c3d24b1b31f31d0c9a5d3eaa4b53193f53dff1bdfed86980d7567a6e195e8d
.pes/objects/cd/1b8e9268619e6717bd3d8e05b3faa74fb3b498760a7b1a71946d540494be0d
.pes/objects/cf/2869af41df6e4165421813f19114bab23848b975af23b6c9f392edc1f6b3f3
.pes/objects/d8/d91acc60cbc6e2d9e68be4bbcc23894138930071d4961faacd08d2a420f8b75
.pes/objects/d8/e4ec6265388041deed285bbe5b1b7d064889d7540c90585caac7b801d3c8f
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e2/1eedd5c26e0031e54571c58359daca3a6dd278e9bbb9a02f94f98fa518c85f
.pes/objects/e2/46e35bf77689ddbdae1fc882a3a87cdf6179a0c3e82eb358d73e204d1947d4
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/ee/6b39aca284120268706239a43427aa7dd2e23687e03e1f2f059ebad02f6497
.pes/objects/ef/536219f5639f733ecaad5810fe03eb8529daf59871cb31d176313a4d78a23
.pes/objects/f2/250085e10ef5db59e425e673796108cf2c059a49e6f7451177cd2dd75f70b3
.pes/objects/f7/7c03788f0e18ff8b43c8134f449b8f19b890cc74faf3329e4c9bb978aefc03
.pes/objects/ff/fce6fc72d75bdb0c369f7723ecc76fbef222b4c4ea0f4f61084bc5c5ab7152
.pes/refs/heads/main
```

```
abheetha@annanya:~/PES1UG24AM045-pes-vcs1$ cat .pes/refs/heads/main
ab92c228589c11102bd2b4c47f866ab323f91bac59bd992355197e938e267ab7
abheetha@annanya:~/PES1UG24AM045-pes-vcs1$ cat .pes/HEAD
ref: refs/heads/main
```

Q5.1: Implementing pes checkout branch

To implement the pes checkout branch, the following changes must happen in .pes/. HEAD must be updated to point to the new branch, for example ref: refs/heads/branchname. If the branch does not exist, create .pes/refs/heads/branchname with the current commit hash.

In the working directory, read the target branch commit hash from its ref file, load that commit tree object, and for every file in the tree write its blob contents to the working directory. Remove any files that exist in the current tree but not in the target tree. Update the index to match the new tree.

What makes this complex is that we must handle files that exist in one branch but not the other. we must detect and refuse checkout if there are uncommitted changes that would be overwritten. Subdirectories with nested trees require recursive handling. The operation must be atomic because a crash halfway through leaves the repo in a broken state.

Q5.2: Detecting Dirty Working Directory Conflicts

To detect conflicts without any extra tools, compare three things: the index, the object store, and the working directory. For each file tracked in the index, read its stored blob hash from the object store. Then read the actual file from the working directory and compute its hash. If

the hashes differ, the file has been modified but not staged. Next, check if that same file differs between the current branch tree and the target branch tree. If both conditions are true, meaning the file is locally modified and differs between branches, checkout must refuse with an error. This way we detect conflicts using only the index and object store with no extra metadata needed.

Q5.3: Detached HEAD and Recovery

When HEAD contains a commit hash directly instead of a branch reference, new commits are still created normally and each new commit points to the previous one as its parent. However, no branch pointer is updated. This means once we switch away, there is no ref pointing to those commits and they become unreachable. They will not show up in `git log` and will eventually be deleted by garbage collection.

To recover those commits, the user must know the commit hash, either from memory or from a reflog if one exists. Once they have the hash they can create a new branch pointing to it by writing the hash into `.git/refs/heads/newbranch`. This re-attaches the commit chain to a named branch making it reachable again.

Q6.1: Garbage Collection Algorithm

The algorithm works in two phases, mark and sweep. In the mark phase, start from all branch refs in `.git/refs/heads/` and HEAD. For each ref, read the commit it points to and add its hash to a reachable set. Then follow the parent pointer to the previous commit and repeat until there is no parent. For each commit, also mark its tree hash as reachable. For each tree, recursively mark all blob hashes and subtree hashes as reachable.

In the sweep phase, walk every file under `.git/objects/` and check if its hash is in the reachable set. If it is not, delete it.

The best data structure for the reachable set is a hash set because it allows $O(1)$ lookup and insertion.

Q6.2: Race Condition Between GC and Commit

The race condition happens like this. A commit operation starts and writes a new blob object to the store. Before it writes the tree object that references that blob, GC runs and scans the object store. GC does not see any tree or commit pointing to the new blob so it marks it as unreachable and deletes it. Now when the commit operation tries to write the tree referencing that blob, the blob is gone and the repository is corrupted.

Git avoids this by using a grace period. GC only deletes objects that are older than a threshold, usually two weeks. Any object written recently is kept even if it appears unreachable. This ensures that in-progress operations have enough time to complete and reference their newly written objects before GC can touch them. Git also uses lock files during critical operations to prevent GC from running at the same time as a commit.